Predator

A Trainable Automated Theorem Prover

Measuring a Learned Policy Against Brute Force

Without Rewarding It for Giving Up

Version 001

Brian Tenneson
M.S., Applied Data Science
Currently Unaffiliated
email: btenneson2301@baypath.edu

July 27, 2026

Predator-ATP-Brian_Tenneson_001 | Version 001 | July 27, 2026
Companion program: `predator.py`

https://btenneson2301.substack.com

Abstract

Predator is a tagged automated theorem prover that learns its search policy from proofs. It is a semi-ideal computer over a formal system $F = (x, y, z)$, carrying a designated formal name $\langle$Predator_1$\rangle$, and it differs from the canonical rule-enumeration machine in exactly one respect: where brute force adjoins every direct consequence at each stage, Predator scores the available consequences with a fitted model and adjoins only the best few.

The evaluation is the substance of this paper. A prover is trained on the first p fraction of a corpus and tested on the remaining $1 - p$, split by corpus order so that training never sees a theorem postdating a test target. The natural way to score it — how far its search effort falls below brute force — is unusable on its own, because a prover that abandons a target expands almost nothing and so scores perfectly. Predator is therefore scored on two numbers that cannot be optimized separately: the *solve rate* over all held-out targets, and the *effort ratio* over targets that brute force and Predator both solved.

The distinction is not hypothetical. At $p = 0.3$ Predator records an effort ratio of 0.172, an apparent speedup of 11.6×, while solving only 70% of held-out targets: a single-number score would rate it a success. At $p = 0.9$ it solves every target at a ratio of 0.085, a genuine 14.7×. Only the paired reporting distinguishes these.

A third quantity is checked rather than optimized. By the depth-floor theorem no policy can reach a target at a stage below its closure depth, so the harness records any breach as an implementation fault. None occurred in any run reported here.

Keywords: automated theorem proving, learned search policy, beam search, brute force baseline, solve rate, effort ratio, temporal split, tagged ATP, closure depth, semi-ideal computer.

Contents

1 What Predator is

Definition 1.1 (Predator as a tagged ATP)**.** Fix a formal system $F = (x, y, z)$ and hypothesis set Γ. *Predator* is the pair (C, H) over Y given by

$$C(\Gamma, 1) = \Gamma, \qquad C(\Gamma, m+1) = C(\Gamma, m) \ \cup \ \mathrm{top}_\beta(\sigma_\varphi, \ D_F(C(\Gamma, m)) \setminus C(\Gamma, m)),$$

$$H(\Gamma, m) = 1 \iff \varphi \in C(\Gamma, m),$$

where φ is the target, σ_φ is a learned score on candidate formulas, β is the beam width, and top_β selects the β highest-scoring candidates. Predator carries a designated formal name $\langle \mathrm{Predator\_1} \rangle$ in the sense of the tagging framework.

Proposition 1.2. *Predator is a semi-ideal computer, and it is a rule-driven target-search ATP for φ whenever $\beta \geq 1$.*

Proof. Stage one is the input by construction. The stage map is nondecreasing because each stage adjoins a set to the previous state. Halting is persistent, since $\varphi \in C(\Gamma, m)$ and monotonicity give $\varphi \in C(\Gamma, m+1)$; and the state freezes on halting if the search stops on success. Rule-drivenness holds because every formula adjoined lies in $D_F(C(\Gamma, m))$, hence is a direct consequence of the current state. Clause (1) of the target-search definition is immediate from the form of H. □

Remark 1.3 (Where the learning lives, and what it cannot touch)*.* Predator is a nondeterministic proof-search policy whose choice function was fitted to data rather than fixed. Its only departure from the canonical machine is the top_β operator: brute force takes all of $D_F(C(\Gamma, m))$, Predator takes β of it. Everything provable about policies therefore applies, in particular the depth floor $p_m \subseteq D_F^{m-1}(\Gamma)$: Predator can reduce how much of each closure round it materializes, and cannot reduce how many rounds are required. Discarding candidates is also why Predator can fail where brute force succeeds, which is the whole reason solve rate must be reported.

2 Training

Definition 2.1 (Split)**.** Order the corpus by the order in which statements were derived. For $p \in (0, 1)$, the first $\lfloor pN \rfloor$ theorems are training data and the rest are held out. The split is by corpus order, not at random: a random split would let the model train on theorems postdating its test targets, which is the usual way such numbers are inflated.

Definition 2.2 (Supervision)**.** For a training target τ, the positive examples are the statements in $\mathrm{anc}(\tau)$, those actually used in its proof; the negatives are statements available before τ that its proof did not use. The model therefore learns to recognize *this is on the way to that*, which is the judgement a search policy has to make at every step.

The score is a logistic model on eight cheap syntactic features of a pair (goal, candidate): token overlap with the goal, the share of the candidate the goal covers, whether the candidate occurs literally in the goal, size mismatch, candidate size, the candidate's closure depth, and whether the candidate is no larger than the goal. Features are standardized before fitting and the training transform is reapplied at scoring, never refitted on the test set.

3 Scoring a prover

Remark 3.1 (Why one number will not do). The intuitive score is the distance of Predator's search effort below brute force. As a sole criterion it is unusable, because it is maximized by failure: a prover that expands three nodes and abandons the target has an effort ratio near zero. Any single-number version of "further from brute force is better" ranks a prover that gives up above one that works.

Definition 3.2 (The two numbers)**.** Let T be the held-out targets, and for each $t \in T$ let $e_{\mathrm{bf}}(t)$ and $e_{\mathrm{pr}}(t)$ be the number of formulas each prover adjoins before reaching t, within a fixed node budget. Write S_{bf} and S_{pr} for the targets each solves. Then

$$\text{solve rate} = \frac{|S_{\mathrm{pr}}|}{|T|}, \qquad \text{effort ratio} = \operatorname*{mean}_{t \in S_{\mathrm{bf}} \cap S_{\mathrm{pr}}} \frac{e_{\mathrm{pr}}(t)}{e_{\mathrm{bf}}(t)}.$$

The ratio is taken over *jointly solved* targets only. Restricting it there is what stops abandonment from counting as efficiency, and reporting it without the solve rate beside it restores the defect.

Definition 3.3 (The floor check)**.** For each jointly solved target the harness compares the stage at which each prover halts. By the depth floor a policy cannot halt earlier than $\delta_F(\Gamma, \varphi) + 1$, so a run in which Predator halts at a strictly earlier stage than brute force indicates a fault — most often a target reachable by a rule the baseline does not implement. This is checked, not optimized.

4 Results

The corpus is generated by running the canonical machine on a propositional Hilbert system with modus ponens and two axiom schemes read as generating rules: 789 statements over 6 stages, with stage sizes $3, 39, 111, 381, 579, 789$. Beam width 8, node budget 60,000, 60 held-out targets of closure depth at least 2.

p	train	held out	brute force solve rate	Predator solve rate	jointly solved	effort ratio mean / median	speedup
0.3	235	551	100%	**70.0%**	42	0.172 / 0.115	11.6×
0.5	393	393	100%	**76.7%**	46	0.132 / 0.101	11.4×
0.7	550	236	100%	100%	60	0.133 / 0.095	10.7×
0.8	628	158	100%	100%	60	0.115 / 0.087	12.5×
0.9	707	79	100%	100%	60	**0.085 / 0.067**	**14.7**×

Depth-floor breaches: 0 in every row.

Remark 4.1 (The metric earning its keep). The first two rows are the argument for Definition 3.2. At $p = 0.3$ Predator's effort ratio is 0.172 and its apparent speedup 11.6× — numbers that, alone, describe a working prover. Its solve rate is 70%. The ratio is respectable precisely because it is computed over the 42 targets Predator did not abandon; the 18 it walked away from contribute nothing to it. A single-number score would have reported the $p = 0.3$ model as roughly as good as the $p = 0.9$ model, when in fact one solves every target and the other misses three in ten.

Remark 4.2 (More training data helps twice). Solve rate rises from 70% to 100% between $p = 0.3$ and $p = 0.7$ and stays there; the effort ratio then continues to improve, from 0.133 at $p = 0.7$ to 0.085 at $p = 0.9$. The two gains are distinct: the first is the model learning to keep the right candidates at all, the second is its learning to rank them higher once kept. Note that held-out set size shrinks as p grows, so the $p = 0.9$ row rests on 79 available targets; the improvement is real but is measured on less.

5 Scope

The corpus is a propositional system of 789 statements generated from three atoms under three rules, and its regularity far exceeds real mathematics. The features are eight hand-built syntactic quantities under logistic regression. The baseline is unguided closure, which is the correct benchmark for this comparison but is a low bar in absolute terms. Nothing here is a result about theorem proving; it is evidence that the prover, the split and the scoring behave as designed, on data where ground truth is exactly known.

The next step is the same harness against a Metamath corpus, where premises are real, proofs are human-written, and lemma reuse means proof lengths are measured in a compiled system rather than the base one.

6 Using it

```
# train on 90% of the corpus, evaluate on the rest
python3 predator.py -p 0.9 --beam 8 --n-test 60 --budget 60000

# sweep the training fraction
for p in 0.3 0.5 0.7 0.9; do python3 predator.py -p $p --no-artifacts; done
```

Each run writes a directory stamped to the minute containing `manifest.json`, `run.log`, `results.json`, `nodes.csv`, `edges.csv` and `predator_1.json` — the last being the fitted prover itself: tag, beam width, weights and the standardization it was trained under. That file is the tagged copy: a serialized name for the machine, reloadable with `Predator.from_dict`, and small enough to print.

Remark 6.1 (A note on the tagged copy). Storing the prover as ⟨Predator_1⟩ together with its weights makes the self-reference machinery available: a copy is a syntactic object denoting the machine, transportable and comparable, and two Predators trained on different corpora can be compared as objects rather than only by their scores. Nothing in this paper needs that, but it is why the serialization records the tag rather than only the parameters.

Acknowledgements

The author thanks AI assistants used as engineering and verification tools during preparation, in particular for identifying that a single-number distance-from-brute-force criterion is maximized by abandoning targets, and for locating a generation-order mismatch that had made better than half the corpus appear underivable. All claims, interpretations and final decisions remain the responsibility of the author.

References

[1] Tenneson, B. (2026). *ML–SIC–ATP Theory: Learned Search Policies for Proof Search*, Version 001.

[2] Tenneson, B. (2026). *Depths of a Simulation and Formalized Self-Awareness*, Merged Edition.

[3] Megill, N. D., & Wheeler, D. A. (2019). *Metamath: A Computer Language for Mathematical Proofs* (2nd ed.). Lulu Press.

[4] Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.